

1 Key Concepts

1.1 Formal Languages

Definition 1 (alphabet). An *alphabet* is a *nonempty, finite* set of symbols.

Definition 2 (string). A *string* over an alphabet Σ is a *finite, ordered* sequence of symbols from Σ .

The set of *all* (finite-length) strings over an alphabet Σ is denoted Σ^* . (The “star” superscript $*$ is known as the Kleene star operator.)

Definition 3 (language). A *language* L over an alphabet Σ is a subset of Σ^* , i.e., $L \subseteq \Sigma^*$. In other words, it is a set of (finite-length) strings over Σ .

1.2 Finite Automata

Definition 4 (DFA). A *deterministic finite automaton* (DFA) is a five-tuple

$$M = (\Sigma, Q, q_0, F, \delta),$$

where:

- Σ is the (finite) input alphabet; an input to the automaton is any (finite) string over this alphabet (i.e., an element of Σ^*).
- Q is the finite set of the automaton’s states.
- $q_0 \in Q$ is the initial state.
- $F \subseteq Q$ is the subset of accepting states. If, after processing the entire input string, the machine ends in one of these states, it is said to *accept* the string; otherwise, it is said to *reject* the string.
- δ is the transition function, which maps a state and input symbol to a next state:

$$\delta: Q \times \Sigma \rightarrow Q.$$

(Recall that $Q \times \Sigma$ is the set of all pairs whose first component is a state in Q , and whose second component is a symbol in Σ .) This function defines the automaton’s transitions as follows: if it is in state $q \in Q$ and reads the next input symbol $\sigma \in \Sigma$, then it transitions to the next state $\delta(q, \sigma) \in Q$. We emphasize that δ must be a *function*, i.e., for every state-symbol pair, there must be *exactly one* next state.

Definition 5 (Language of a DFA). The *language of a finite automaton* M is the set of strings that the automaton accepts: $L(M) = \{x \in \Sigma^* : M \text{ accepts } x\}$. We also say that M *decides* this language: it accepts every string in $L(M)$, and rejects every string not in $L(M)$.

1.3 Turing Machines

Definition 6 (Turing Machine). A Turing machine is a seven-tuple

$$M = (\Sigma, \Gamma, Q, q_{\text{start}}, q_{\text{acc}}, q_{\text{rej}}, \delta)$$

whose components are as follows:

- Σ is the (finite) input alphabet. An input to the machine is any (finite) string over this alphabet (i.e., an element of Σ^*).
- Γ is the (finite) tape alphabet, which must contain Σ as a subset (i.e., $\Sigma \subseteq \Gamma$), in addition to a special blank symbol $\perp \in \Gamma$ that is not in the input alphabet (i.e., $\perp \notin \Sigma$). It may contain other elements as well. The alphabet Γ is the set of symbols that may be written to the cells of the tape.
- Q is the finite set of states.
- $q_{\text{start}} \in Q$ is the initial state.
- $q_{\text{acc}}, q_{\text{rej}} \in Q$ are, respectively, the accept state and reject states, which must be distinct. Together, they comprise the set of final states $F = \{q_{\text{acc}}, q_{\text{rej}}\}$. If the machine ever transitions to either of these states, then the computation *immediately* halts, and the machine is said to *accept* or *reject*, respectively.
- δ is the transition function. Its input is a non-final state and a tape symbol (as read by the head), and its output is a new state, a new tape symbol (to be written by the head), and a direction for the head to move (left or right). Formally, it is a function

$$\delta: \underbrace{(Q \setminus F)}_{\text{Non-terminal state}} \times \underbrace{\Gamma}_{\text{Cell contents}} \rightarrow \underbrace{Q}_{\text{Next state}} \times \underbrace{\Gamma}_{\text{Symbol to write}} \times \underbrace{\{L, R\}}_{\text{Where to move the head}},$$

where L and R represent “left” and “right”, respectively.

A Turing machine M runs on an input x as follows. Initially, the characters of x appear in order from the beginning of the tape (one character per cell), and the tape head points to the first cell of the tape. The machine starts in the initial state q_0 . The machine then repeatedly applies its transition function (each time using its current state and the symbol read by the tape head to write a symbol, move the head, and transition to the next state), until it transitions to one of the final states (if it ever does so).

Unlike a DFA, the behavior of a Turing machine M when run on an input $x \in \Sigma^*$, denoted $M(x)$, falls into exactly one of *three* cases:

1. $M(x)$ eventually *accepts* (makes a transition to the accept state q_{acc});
2. $M(x)$ eventually *rejects* (makes a transition to the reject state q_{rej});
3. $M(x)$ runs forever, or “*loops*” (it never makes a transition to a final state).

Church-Turing Thesis. This posits that a function can be computed by an “effective” or “mechanical” procedure—in other words, an algorithm—if and only if it can be computed by some Turing machine. It is not a theorem, but a statement of belief (or an axiom) about what “computation” is, and that it is captured by the Turing-machine model.

Definition 7 (Language of a Turing machine). The *language* $L(M)$ of a Turing machine M is the set of strings that the machine accepts:

$$L(M) = \{x : M \text{ accepts } x\}$$

Definition 8 (‘Decides’ for Turing machines). A Turing machine M *decides* a language $L \subseteq \Sigma^*$ if:

1. M accepts every $x \in L$, and
2. M rejects every $x \notin L$.

Equivalently:

1. M halts (accepts or rejects) on every $x \in \Sigma^*$, and
2. $x \in L$ if and only if $M(x)$ accepts.

A language is *decidable* if some Turing machine decides it.

Definition 9 ('Recognizes' for Turing machines). A Turing machine M *recognizes* a language $L \subseteq \Sigma^*$ if:

1. M accepts every $x \in L$, and
2. M *rejects or loops* on every $x \notin L$.

Equivalently: $x \in L$ if and only if $M(x)$ accepts.

A language is *recognizable* if some Turing machine recognizes it.

2 Discussion Problems

2.1 Decision Problems and Languages

1. Determine whether each of the following problems is a decision problem. If it is, formulate a language containing exactly the "yes" instances of the problem (make sure to specify how you would encode the elements of the language). If it is not, explain how you could change it into a decision problem.

- (a) Given a set of cities, find a set of three cities whose name share at least 3 letters.

Solution: This not a decision problem, as the problem is not asking for a yes/no answer. We could change it to a decision problem by rewording it to: *Given a set of cities, is there a set of three cities whose name share at least 3 letters?*

- (b) Given a list of classes (specified by (s_c, e_c) where s_c is the start time of class c and e is the end time of class c), is it possible to enroll in at least three classes without having any overlaps in lecture times?

Solution: This is a decision problem. Elements of the language could be encoded using the natural encoding for an array of pairs of integers (where each pair is two integers with a separator character in between them). The language would be $\{\langle L \rangle : \exists c_1, c_2, c_3 \in L \text{ and } c_1, c_2, \text{ and } c_3 \text{ do not overlap}\}$.

2. For each of the following decision problems, (i) define a reasonable (finite) alphabet Σ , (ii) give the encoding (over the alphabet) of a representative input object, and (iii) define a language L for the corresponding decision problem. Use the notation $\langle X \rangle$ for the encoding of object X as a string over the alphabet.

- (a) Given a binary string b , does it have an even number of 0s?
- (b) Does a given non-negative integer x , when written in base 10, have 3 as the last digit?
- (c) Does a given C++ program, when run on input “eecs376”, halt within 376 steps?

Solution:

- (a)
 - i. $\Sigma = \{0, 1\}$.
 - ii. For a binary string b , write it as a string of bits. For example, if b is the binary string “010”, then $\langle b \rangle = 010$.
 - iii. $L = \{\langle b \rangle : b \text{ is a binary string with even number of 0s.}\}$
- (b)
 - i. $\Sigma = \{0, \dots, 9\}$.
 - ii. For an integer k , write it in base 10 in the usual way, as a string of digits. For example, if k is the integer forty-seven, then $\langle k \rangle = 47$.
 - iii. A corresponding language would be $L_{\text{EndsWith3}} = \{\langle k \rangle : k \bmod 10 = 3\}$. It would also be acceptable to copy the phrasing of the decision problem, i.e., $L_{\text{EndsWith3}} = \{\langle k \rangle : k \text{ written in base 10 has 3 as the last digit}\}$.

- (c)
 - i. We can simply use the ASCII character set.
 - ii. We can encode this program directly as an ASCII string, including all newlines, tabs, etc. (which are ASCII characters).

```
#include <string>
```

```
int main(int argc, char** argv) {  
    if (std::string(argv[1]) == "eecs376")  
        return 1;  
    return 0;  
}
```

- iii. We can express the language as

$L_{\text{HaltEECS376}} = \{\langle M \rangle : M \text{ halts when run on input “eecs376”, within 376 steps}\}$.

Here ‘steps’ can have any reasonable definition: assembly instructions of the compiled C++ program, state transitions of a Turing machine that implements the program, etc.

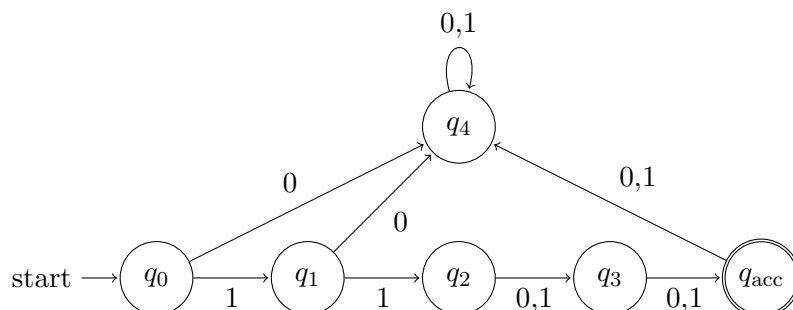
2.2 DFAs

3. True or False: there is a DFA that decides the following language:

$\{x : x \text{ is a binary string of length 4 that starts with two 1s}\}$.

Justify your answer.

Solution: True. We show this by constructing a DFA, then proving that this DFA decides the given language. Consider the following:



We will prove this DFA decides the language by considering each case. If a binary string has a length of less than 4, the DFA cannot reach state q_{acc} , as it is at least 4 nodes away. If a binary string has a length of more than 4, it will always reach q_4 , and be stuck there, as every path > 4 steps ultimately leads to q_4 . If a binary string has a length a 4 and starts with 11, then it will end with q_{acc} . If it starts with 00, 01, or 10, it will end at q_4 . In every case, our DFA decides correctly.

4. True or False: there is a DFA that decides the following language: $\{x : x \text{ is a palindrome}\}$, where the alphabet Σ has at least two elements. Justify your answer.

Solution: False. (However, note that the statement is actually trivially true if the alphabet has only one element, because in that case *every* string is a palindrome! The proof below will critically use the hypothesis that the alphabet has two distinct elements.)

Typically, if deciding a language would seemingly require a DFA to “remember” or “count” an arbitrarily large part of the input string, it is a clue that this language cannot be decided by a DFA. In the case of palindromes, “remembering” the first half of the input string, so that it can be compared (in reverse) to the second half, requires unlimited memory, and is not something a DFA can do. However, this is just a heuristic, and we need an actual proof to show that a certain language cannot be decided by a DFA.

To formally prove that no DFA decides the language of palindromes (over an alphabet of at least two characters), we will apply a similar argument to the one we used to prove that no DFA decides the language $\{0^n 1^n : n \in \mathbb{N}\}$ —using the pigeonhole principle.

Let M be some arbitrary DFA. We will show that M does not decide the language. By definition, M must have some finite number of states s . Consider the input string x^{s+1} , where $x \in \Sigma$ is an arbitrary symbol in the alphabet. By the pigeonhole principle, there is some state in M that is repeated as M processes x^{s+1} . Consequently, there must be some distinct indices $0 \leq i < j \leq s + 1$, where M is in the same state after processing x^i and x^j .

Now, consider another symbol $y \in \Sigma$, where $y \neq x$. (Here is where we use the fact that the alphabet has at least two elements.) Let $S_1 = x^i y x^i$, which is a palindrome, and let

$S_2 = x^j y x^i$, which is not a palindrome because $i \neq j$. Since M is in the same state after processing x^i and x^j , and the suffixes of S_1 and S_2 are both $y x^i$, M must give the *same answer* on both S_1 and S_2 —either accept, or reject. Therefore, M answers incorrectly on one of these, so it does not decide the language. Since M was defined to be an arbitrary DFA, we conclude that no DFA decides this language.

2.3 Turing Machines

Are the following true or false?

1. (True/False) A Turing machine can potentially decide more than one distinct language.

Solution: False.

A Turing machine decides either one language, or no language at all.

- By definition, a Turing machine that decides a language must halt on every input, so any Turing machine that loops on some input does not decide a language.
- Now consider a TM M that decides at least one language L . By definition, this means that $x \in L \iff M(x)$ accepts. So, if M also decides language L' , we get that $x \in L \iff M(x)$ accepts $\iff x \in L'$, which means that $L = L'$ (they are the same language). Hence, M decides exactly one language.

In fact, this proof also shows that *every* Turing machine *recognizes* exactly one language, because for the second part of the argument we did not use the fact that M halts. We only used the fact that $x \in L \iff M(x)$ accepts, which is the definition of “ M recognizes L .”

2. (True/False) For any decidable language L , there is more than one distinct Turing machine M that decides L .

Solution: True.

Consider an arbitrary decidable language L and an arbitrary TM M that decides it. We can construct a modified machine M' that has two extra states: it begins by transitioning one cell right, then one cell left (not changing either character it reads), then transitions to the start state of an identical state machine as M . This M' is a different machine from M , and it also decides L (because the initial two head movements and transitions do not affect the ultimate behavior of the machine). In fact, we can generalize this modification to any finite number of moves right, and then back to the start. (Other modifications work as well, like adding ‘unreachable states’ that have no incoming transitions.) So, for any decidable language, there are *infinitely many* Turing machines that decide it.

3. (True/False) Any Turing machine recognizes some language.

Solution: True.

Any Turing machine M has some (possibly empty) set of input strings that it accepts; recall that this is called the language of the machine, denoted $L(M)$. That is, $x \in L(M) \iff M(x)$ accepts. This is the definition of “ M recognizes $L(M)$.”

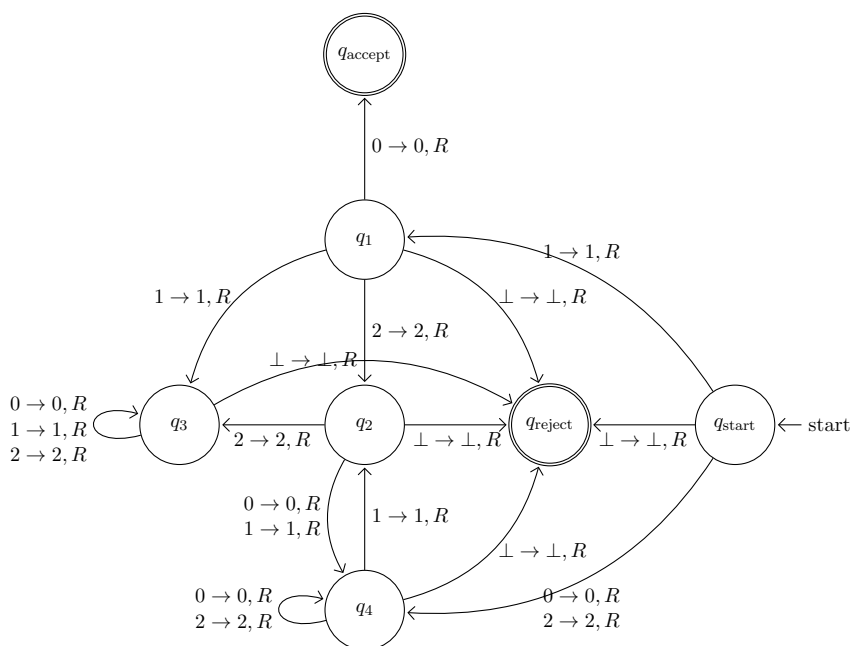
4. (True/False) There exists a language X that is the language of some three-tape Turing machine, but is not the language of any two-tape Turing machine.

Solution: False.

We have previously seen in lecture that 1-tape and 2-tape Turing machines have equivalent computational power: the accept/reject/halt behavior of any 1-tape machine can be emulated by some 2-tape machine (on all inputs), and vice versa.

An analogous argument shows that 2-tape and 3-tape Turing machines have equivalent computational power as well. Therefore, the language of an arbitrary 3-tape machine—i.e., the set of inputs that the machine accepts—is also the language of some 2-tape machine (and vice versa).

5. Consider the Turing Machine whose state diagram is given below:



Which of the following statements is true about this Turing Machine?

- (a) It accepts all strings that have the substring “10.”
- (b) It accepts all strings that start with the substring “10.”
- (c) It loops on any input string consisting only of 2s.

- (d) The string “10 $\perp$ 1” is in the language of this Turing Machine.

Solution: Only B is true.

A is false. One counter-example is the string “010” which contains the substring “10” but which causes this machine to reject.

B is true. Any string that begins with “10” causes this machine to go to the accept state, so the machine accepts it regardless of the remaining characters of the string.

C is false because on an input string consisting of only 2s, the machine will eventually reach a blank symbol, then reject.

D is false. The input string can only contain symbols from the input alphabet Σ , and the special symbol $\perp$ is not in Σ by definition. So any string that contains $\perp$ is not in the language.

2.4 Decidability

- Let A and B be languages. $A \triangle B$ denotes the *symmetric difference* of the sets A and B . That is, $A \triangle B = (A \cup B) \setminus (A \cap B)$. This can also be seen as the set of strings that are in *exactly one* of A or B .

Suppose that A and B are decidable. Then $A \triangle B$ is decidable for (all/some/no) such A, B . Provide justification to support your answer.

Solution: $A \triangle B$ is decidable in **all** cases. We show this by describing a TM D that decides $A \triangle B$, using Turing machines M_A, M_B that respectively decide A, B (which exist by hypothesis). Specifically, $D(x)$ runs both $M_A(x)$ and $M_B(x)$, and accepts if exactly one of them accepts; otherwise, it rejects.

Analysis:

Observe that

$$\begin{aligned} x \in A \triangle B &\implies x \in (A \cup B) \setminus (A \cap B) \\ &\implies (x \in A \setminus B) \vee (x \in B \setminus A) \\ &\implies \text{exactly one of } M_A \text{ or } M_B \text{ accepts} \\ &\implies D(x) \text{ accepts.} \end{aligned}$$

On the other hand,

$$\begin{aligned} x \notin A \triangle B &\implies x \in (A \cap B) \cup \overline{(A \cup B)} \\ &\implies M_A \text{ and } M_B \text{ both accept, or both reject} \\ &\implies D(x) \text{ rejects.} \end{aligned}$$

Therefore, D decides $A \triangle B$, by definition.

3 Additional Problems

3.1 Decision Problems and Languages

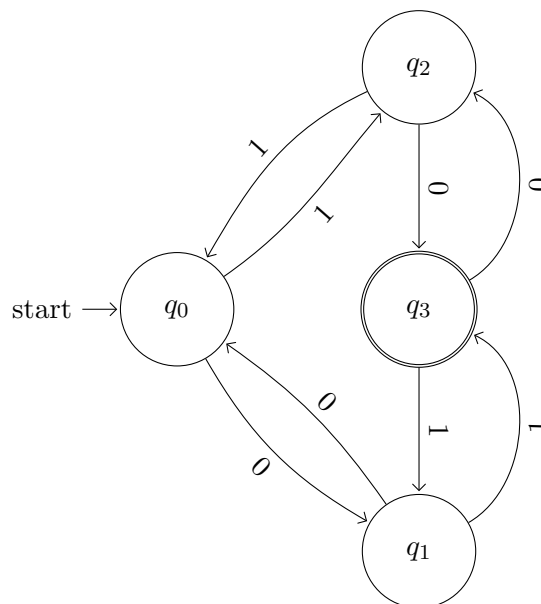
1. The language that represents all positive integers divisible by 4 can be written as:

- (a) $L = \{n \in \mathbb{Z}^+ : n \bmod 4 = 0\}$
- (b) $L = \{n, k \in \mathbb{Z} : n/4 = k\}$
- (c) $L = \{n \in \mathbb{Z}^+ : \gcd(n, 4) = 1\}$
- (d) $L = \{n, k \in \mathbb{Z}^+ : n \bmod 4 = 2\}$
- (e) None of the above

Solution: (A)

3.2 DFAs

1. Consider the following DFA:



- (a) Recall that a DFA is formally defined as a 5-tuple $(Q, \Sigma, \delta, q_0, F)$. State what these components are for the above DFA.

Solution:

Q : This is the set of states, $\{q_0, q_1, q_2, q_3\}$

Σ : This is the set of characters the DFA can read, $\{0, 1\}$

δ : This is the transition function for the DFA,

state	input	next state
q_0	0	q_1
q_0	1	q_2
q_1	0	q_0
q_1	1	q_3
q_2	0	q_3
q_2	1	q_0
q_3	0	q_2
q_3	1	q_1

q_0 : This is the start state; this is often denoted by an arrow labeled 'start'

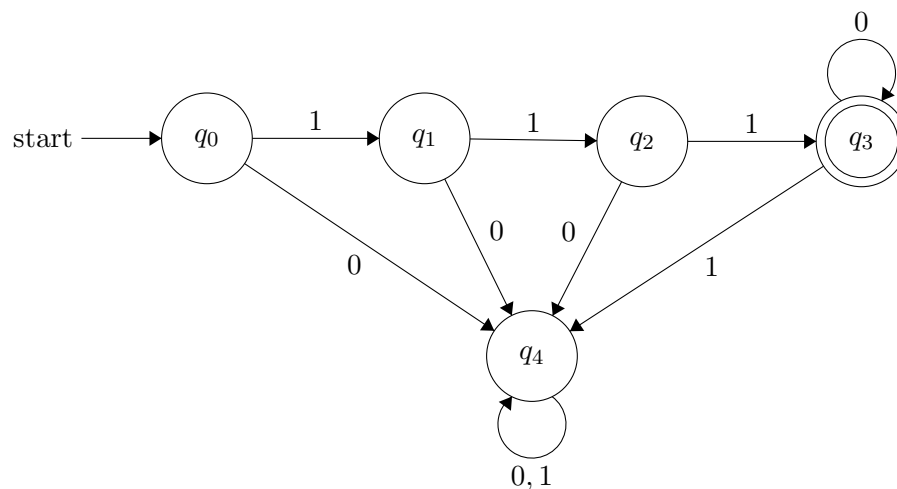
F : This is the *subset* of accepting states, $\{q_3\}$

(b) What language does the above DFA decide?

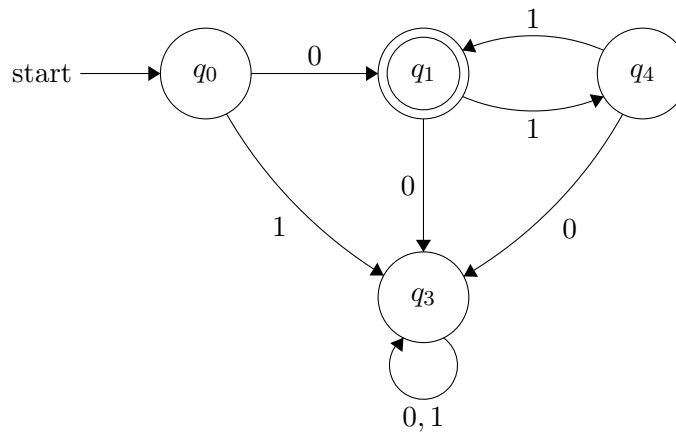
Solution: The language is $\{x : x \text{ has an odd number of 0s and an odd number of 1s}\}$.
Some inputs that the program accepts: 010101, 000001, 111110, 11000001, and inputs that the program doesn't accept: 0, 1, 100, 1100.

2. Draw a DFA that decides the language 1110^* .

Solution:



3. Consider the following DFA, where the alphabet is $\Sigma = \{0, 1\}$.



(a) For each of the following strings, determine whether the string is accepted by the DFA:

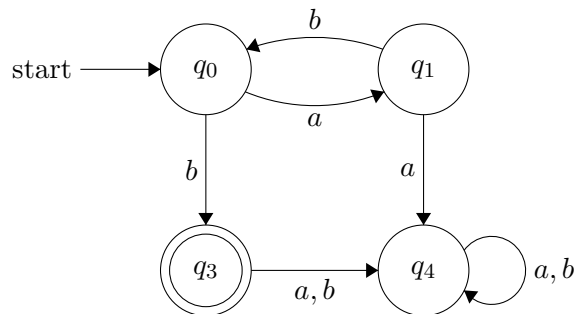
- ε
- 0
- 1
- 010
- 011
- 111

Solution: 0,011 are accepted. ε , 1,010,111 are rejected.

(b) What language does the DFA decide? Provide your answer in ‘string notation’.

Solution: Regular expression: $0(11)^*$.

4. Consider the following DFA, where the alphabet is $\Sigma = \{a, b\}$.



(a) For each of the following strings, determine whether the string is accepted by the DFA:

- ε
- a
- b
- bb

- aab
- $abbabb$
- $ababb$

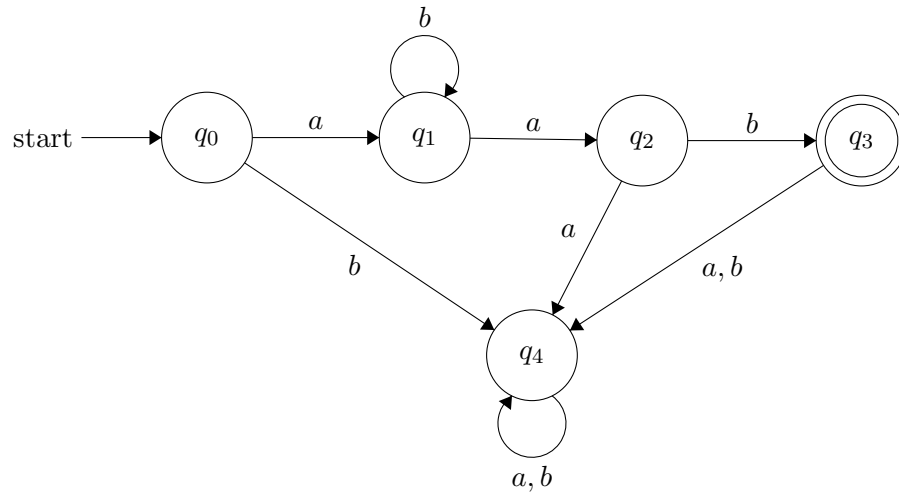
Solution: $b, ababb$ are accepted.

(b) What language does this DFA decide? Provide your answer in ‘string notation’.

Solution: Regular expression: $(ab)^*b$.

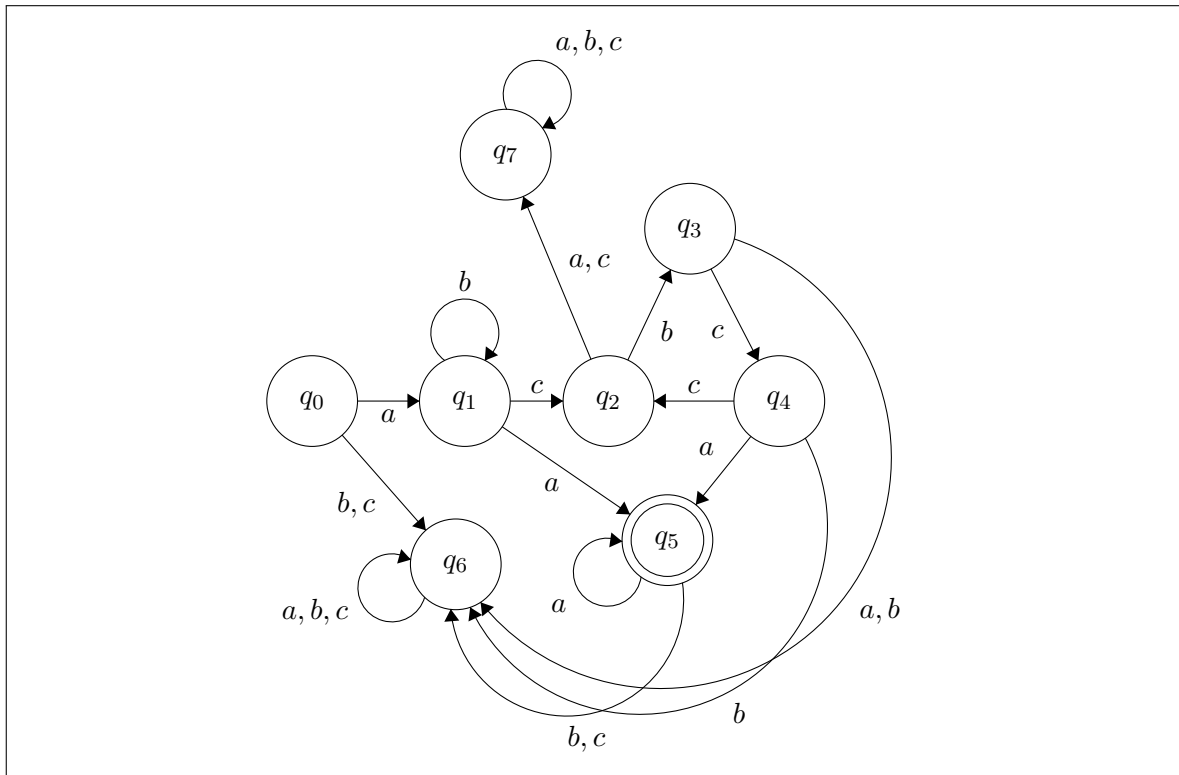
5. Draw a DFA that decides the language ab^*ab .

Solution:



6. Draw a DFA that decides the language $ab^*(cbc)^*aa^*$.

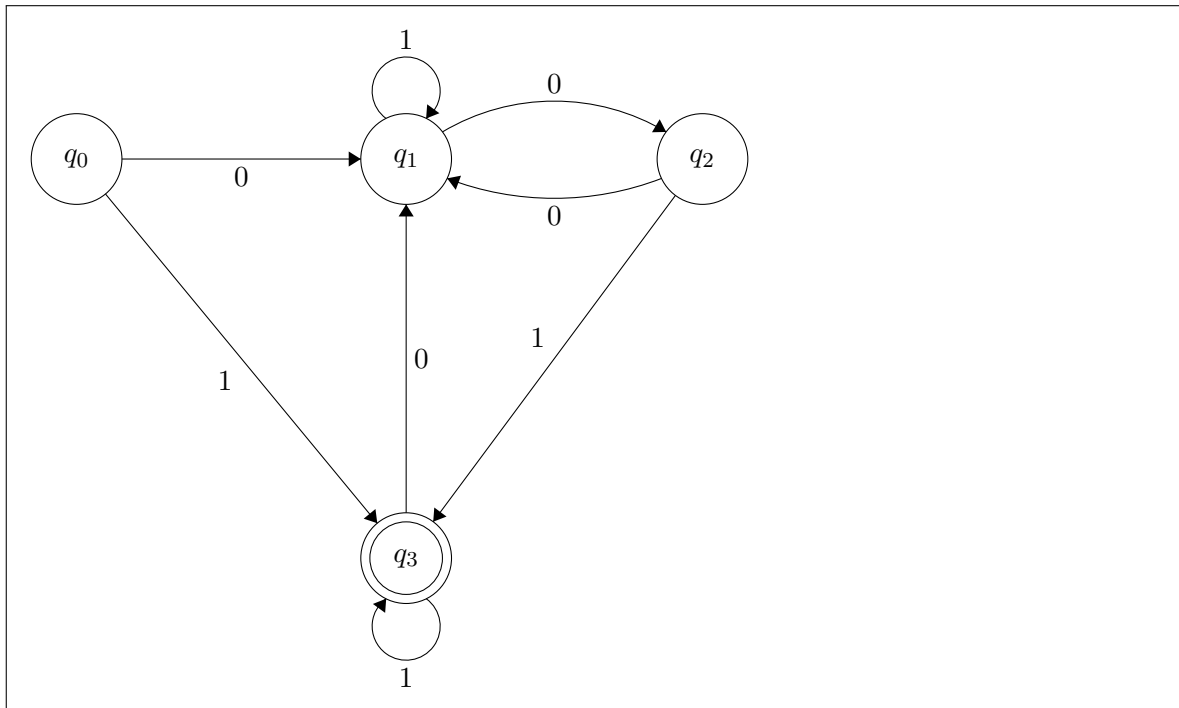
Solution: Note that q_7 is not actually necessary, it was just added so the diagram is neater.



7. Draw a DFA that decides the language

$$\{x \in \{0, 1\}^* : x \text{ has an even number of 0s and the last character of } x \text{ is } 1\}.$$

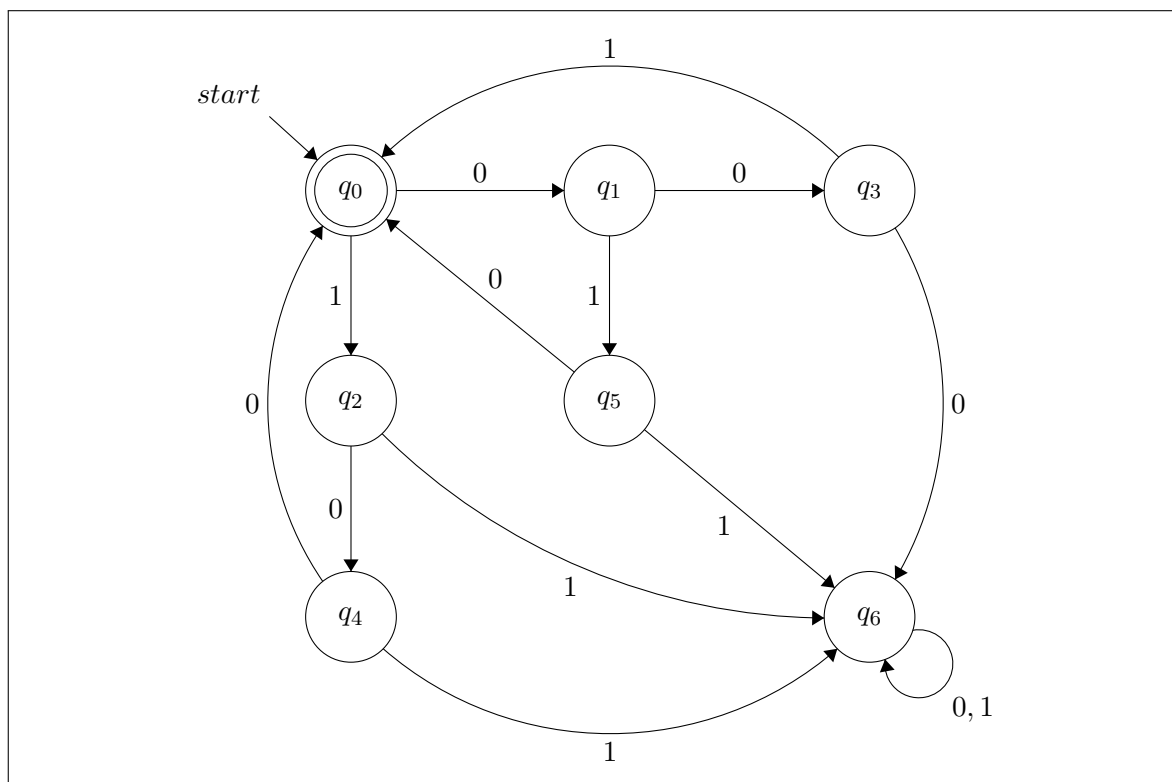
Solution: Every time an odd number of 0s has been read, we are in q_1 ; we can read one more zero to move to q_2 , which represents having read an even number of 0s. We then end up in q_3 (the accept state) whenever the latest input was a 1.



8. Draw a DFA that accepts only the set of binary strings of length $3n$, for some integer $n \geq 0$, that contain exactly two 0's for every "block" of 3 characters.

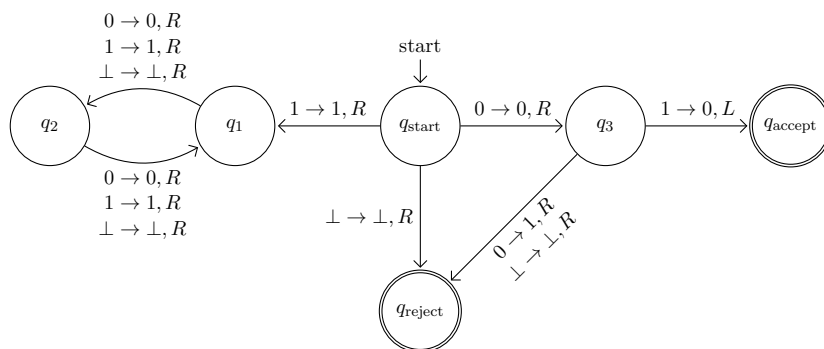
For example, 100010 is in the language because it has two blocks, 100 and 010, which both have *exactly* two 0's. The empty string, ϵ , is also (vacuously) in the language. On the other hand, 00100 is not in the language because its length is not a multiple of 3 so it cannot be split into blocks, and 110001 is not in the language because there is only one 0 in its first block, 110.

Solution:



3.3 Turing Machines

1. Consider the Turing Machine whose state diagram is given below:



What language over $\Sigma = \{0, 1\}$ does this Turing machine decide, if any?

Note: We say that M decides language A if $A = \{w : M \text{ accepts } w\}$ and M halts on all inputs.

- (a) $\{01\} \cup \{s \in \Sigma^* : s \text{ starts with } 1\}$
- (b) $\{01\}$
- (c) $\{s \in \Sigma^* : s \text{ starts with } 1\}$
- (d) $\{\varepsilon, 0, 00\}$
- (e) None of the above

Solution: This TM fails to halt on inputs of the form $1(0|1)^*$, so it does not decide a language.

2. (Every / Some / No) Turing machine writes to only finitely many of its tape cells during its execution.

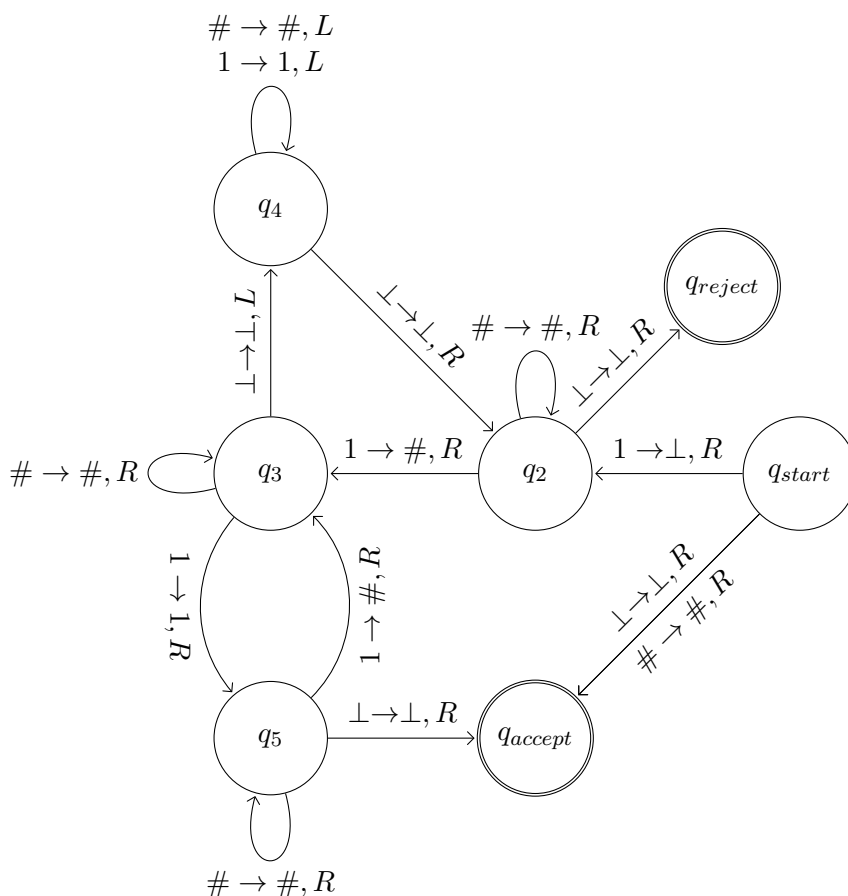
Solution: Some. Finitely Many: this occurs anytime a Turing machine halts on an input. Halting implies that only finitely many transitions occurred, and each transition causes exactly one cell on the tape to be written to (even if what is written back is the same thing as was already written).

Infinitely Many: One example would be a Turing machine that was given the empty-string input and has a transition function including the transition $\delta(\perp, q_0) = (q_0, \perp, R)$. In general, a Turing machine would write to infinitely many tape cells if the Turing machine loops AND the head continues moving to the right with every step.

3. (Every / Some / No) Turing machine that decides a language writes to only finitely many of its tape cells during its execution.

Solution: Every. A TM that decides a language must accept or reject for every input in Σ^* , which means it halts after a finite number of steps, on any input. If its execution is finite, the number of cells it writes to must also be finite, because it writes to only one cell per step.

4. Consider the Turing machine represented by the following figure, for input alphabet $\Sigma = \{1\}$ and tape alphabet $\Gamma = \{1, \#, \perp\}$. (Note that there are *two* extra characters in the latter.)



What language does this Turing machine decide, if any? Justify your answer.

Hint: What strings does it not accept?

Solution: This Turing machine decides the language:

$$\{x : x \notin \{1\}^{2^n} \text{ for } n \in \mathbb{N}_0\}$$

We will consider some example runs of the machine to understand how this Turing machine works, and argue how this example generalizes to any string in $\{1\}^*$.

Consider the input string (which is in the above language) “1111111” and note that the rest of the tape is filled with $\perp$:

- First we will have q_{start} and transition to q_2 when we detect the first 1 giving us the string: “ $\perp$ 1111111”
- We detect a 1 so we transition to state q_3 as follows: “ $\perp$ #111111”
- We now do a loop making every other 1 a # until we hit the end of the string, the result is as follows: “ $\perp$ #1#1#1#”

- We then move to start q_4 reversing directions now and moving left. The function of state q_4 is to take us back to the beginning of the string (the first $\perp$ symbol). After we hit the first $\perp$ symbol we are at q_2 with the following on our tape: " $\perp \#1\#1\#1\#$ "
- We repeat the process again replacing every other 1 with a $\#$ and when we reach the end of the string we have:
" $\perp \#\#\#1\#\#\#$ "
- We reset back to the beginning of the tape and are again at:
" $\perp \#\#\#1\#\#\#$ "
- After another iteration we are at the end of the tape with:
" $\perp \#\#\#\#\#\#$ "
- We skip ever going to q_5 this time since we are already at the end of the tape after one replacement. We transition back to the start to state q_2 :
" $\perp \#\#\#\#\#\#$ "
- At q_2 we loop through the rest of the string until we hit a $\perp$ since it is composed entirely of $\#$ characters now. We notice that if we ever hit a $\perp$ symbol while at state q_2 we transition to state q_{reject} , and thus we have finished, rejecting the string $\{1\}^{2^3}$ as expected.

More generally, each pass of the machine over the input replaces every other 1 with $\#$, and accepts if the number of 1s it encounters is an odd number greater than one. Therefore, the machine rejects if and only if every pass sees an even number of 1s, followed by a final pass with exactly one 1. The machine therefore rejects every string of the form $x \in \{1\}^{2^n}$, and accepts all others.

3.4 Decidability

1. Show the following statement is true: For any decidable language L , the language $L' = L \cup \{\varepsilon\}$ is decidable.

Solution: Let D_L decide L .

There are two cases.

If $\varepsilon \in L$, then $L = L'$ and the claim holds.

If $\varepsilon \notin L$, then we can use the following decider:

```

1: function  $D_{L'}(x)$ 
2:   if  $x = \varepsilon$  then accept
3:   else if  $D_L(x)$  accepts then accept
4:   else reject
```

$D_{L'}$ halts on all inputs because D_L is a TM that decides L .

We have the following implications:

- $x \in L' \implies x \in L \cup \{\varepsilon\} \implies x = \varepsilon \vee x \in L$. If $x = \varepsilon$, then $D_{L'}$ accepts on lines 2-3. If $x \in L$, then $D_L(x)$ accepts on lines 4-5.
- $x \notin L' \implies x \notin L \cup \{\varepsilon\} \implies x \neq \varepsilon \wedge x \notin L$. Because $x \neq \varepsilon$ and $x \notin L$, x satisfies neither of the conditions which would cause $D_{L'}$ to accept, so $D_{L'}$ rejects.

So $D_{L'}$ decides L' .

2. The set difference of S and T is written $S \setminus T$, and defined as $S \setminus T = \{s \in S : s \notin T\}$.
If S and T are both decidable, $S \setminus T$ is (always/sometimes/never) decidable.

Solution: $S \setminus T$ is always decidable.

A TM that decides $S \setminus T$ can be constructed by running a TM that decides S and a TM that decides T and performing the appropriate checks.

```
function  $D_{S \setminus T}(x)$   
  if  $D_S(x)$  accepts and  $D_T(x)$  rejects then  
    accept  
  reject
```

Analysis is nearly identical to the proof above for symmetric difference.